AI Operations Assistant

Panduan Pengguna Lengkap

Dari input sampai output yang diharapkan

Proyek edukasi dengan data sintetis · Python 3.12 · FastAPI
Disusun dari aplikasi yang dijalankan dengan DeepSeek, 4 November 2026

Daftar isi

1. Gambaran umum: apa ini dan untuk siapa
2. Persiapan dan instalasi
3. Alur kerja besar (workflow)
4. Menjalankan aplikasi dan antarmuka web
5. **Dari input sampai output yang diharapkan** (langkah 0 sampai 10, dengan gambar)
6. Referensi API: input, output, how, why
7. Konfigurasi dan memakai provider LLM asli
8. Menjaga kunci rahasia
9. Format error dan penanganannya
10. Evaluasi kualitas retrieval
11. Pemecahan masalah
12. Batasan yang perlu diketahui
13. Referensi dokumen lain

1. Gambaran umum

AI Operations Assistant adalah asisten operasional untuk perusahaan kecil. Ia menjawab pertanyaan dari dokumen dengan sumber, mencari data order lewat agen, dan **selalu menahan setiap perubahan data sampai seorang manusia menyetujuinya**.

> **Penting:** ini proyek edukasi dengan data sintetis. Belum dipakai di produksi, dan baru dievaluasi secara kecil dengan satu LLM asli, yaitu DeepSeek. Tanpa kunci, provider `mock` hanya mengulang teks Anda, tetapi retrieval, tool, dan approval tetap berjalan sungguhan.

Fitur	Fungsi	Kenapa dibuat begini (why)
Tanya dokumen (RAG)	Mencari teks relevan, menjawab hanya dari teks itu, mengembalikan sumber	Jawaban bisa diperiksa. Jika tidak ada yang relevan, model tidak dipanggil dan tidak ada jawaban karangan
Agen dengan tool	Loop buatan sendiri yang mencari dokumen dan membaca order	Setiap langkah bisa dibaca, diuji, dan dibatasi (langkah dan waktu)
Persetujuan manusia	Model hanya bisa mengusulkan perubahan; hanya klik manusia yang menjalankannya	Tidak ada tool yang memungkinkan model menyetujui dirinya sendiri
Workflow tiket	Klasifikasi tiket, cari order, buat draf balasan, tunggu persetujuan	Pengulangan permintaan tidak pernah mengirim dua kali
Bagian yang bisa diganti	LLM, vector store, penyimpanan, dan pengiriman pesan lewat interface	Provider bisa ditukar tanpa mengubah logika
Evaluasi	Kumpulan pertanyaan tetap mengukur retrieval; CI gagal jika skor turun	Kualitas terukur, bukan perasaan
Insert Data	Menambah dokumen dan order; AI hanya mengisi draf formulir	Data masuk tanpa mengubah file proyek, dan orang tetap memeriksa sebelum menyimpan
Settings	Menampilkan provider, memilih model, dan menguji koneksi	Masalah kunci atau saldo terlihat sebelum dipakai. Kunci tidak bisa diisi di sini karena API tanpa login
Antarmuka web	Satu halaman per fitur, tanpa build step	Berjalan di mana pun API berjalan, bahkan offline

Peran pengguna

- **Operator:** bertanya, mengecek order, menyetujui atau menolak aksi yang diusulkan.
- **Reviewer portofolio:** ingin cepat melihat sistem bekerja, memeriksa sumber, tool call, dan hasil evaluasi.

2. Persiapan dan instalasi

Kebutuhan: Python 3.12 dan Git. API key hanya perlu untuk jawaban asli (bagian 7). Tanpa kunci, provider mock menggantikan model.

Windows (PowerShell)

```
git clone https://github.com/maeshataruna99-lgtm/core-ai-automation.git
cd core-ai-automation
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python scripts/demo.py
```

> Jika PowerShell menolak `Activate.ps1`, jalankan dulu `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`. Efeknya hanya untuk jendela itu.

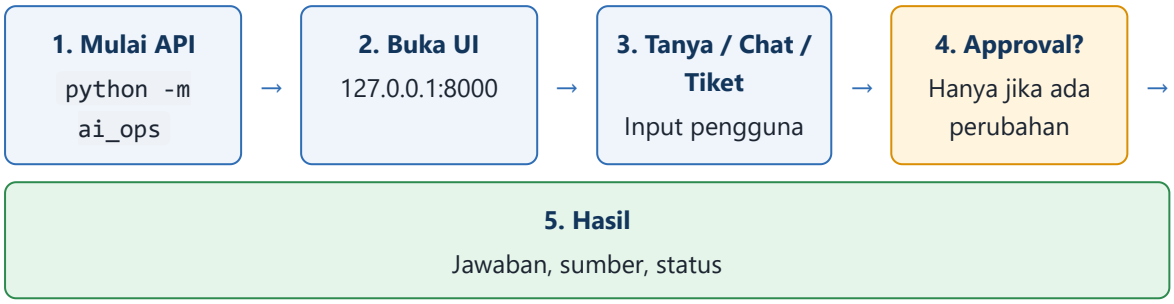
Linux

```
git clone https://github.com/maeshataruna99-lgtm/core-ai-automation.git
cd core-ai-automation
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
python scripts/demo.py
```

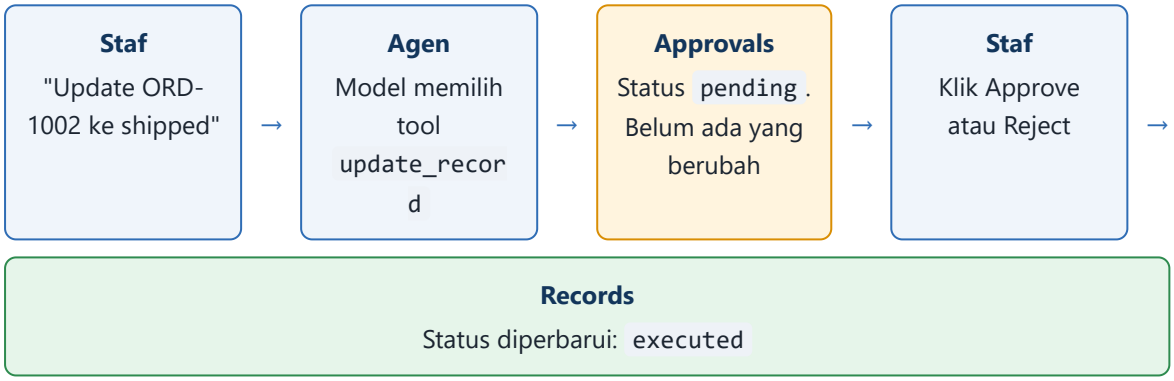
Output yang diharapkan: `scripts/demo.py` menjalankan API, melewati setiap fitur, memeriksa setiap hasil, lalu menghentikan API. Ia tidak mengubah apa pun di repositori.

3. Alur kerja besar (workflow)

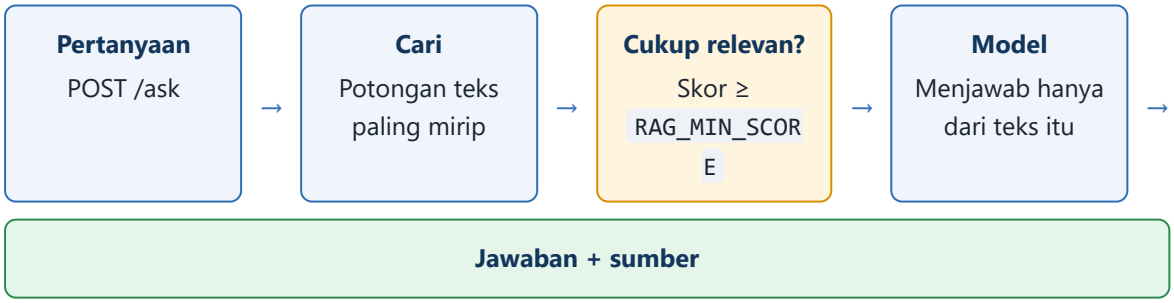
3.1 Alur penggunaan harian



3.2 Aturan terpenting: model hanya mengusulkan

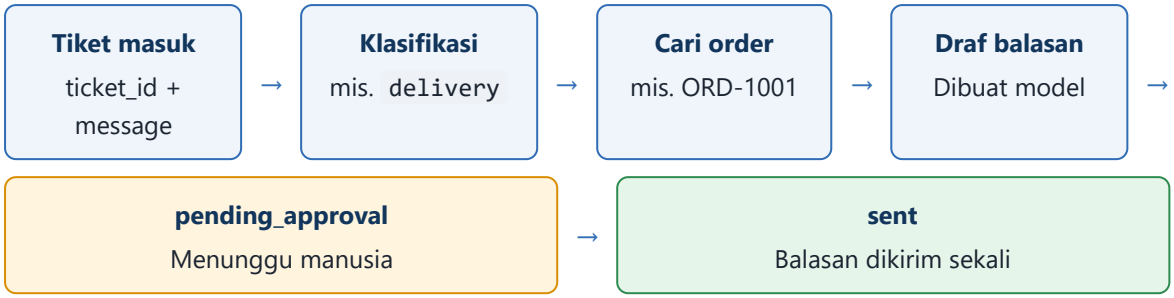


3.3 Alur pertanyaan dokumen (RAG)



Jika jawaban atas "cukup relevan?" adalah tidak, API tetap membalas 200 dengan `found: false`, `sources` kosong, dan model tidak dipanggil.

3.4 Alur tiket



4. Menjalankan aplikasi dan antarmuka web

Jalankan dari **root repositori** supaya dokumen contoh ditemukan.

Sistem	Perintah
Dengan DeepSeek, setelah `.env` diisi	`python -m ai_ops`
Tanpa kunci, Windows (PowerShell)	`$env:AI_OPS_LLM_PROVIDER = "mock"` lalu `python -m ai_ops`
Tanpa kunci, Linux	`AI_OPS_LLM_PROVIDER=mock python -m ai_ops`

Buka `http://127.0.0.1:8000` di browser (otomatis menuju `/ui/`). Hentikan dengan Ctrl+C. Halaman dokumentasi API interaktif ada di `/docs` dan butuh internet untuk tampilannya.

Halaman di antarmuka

Halaman	Input	Output	Kenapa ada
Ask	Pertanyaan	Jawaban berlabel "AI-generated" dan sumbernya, atau "No answer found"	Jawaban bisa diperiksa ke dokumen
Chat	Pesan teks (ada batas panjang dan penghitung)	Jawaban berlabel "AI-generated", sumber, dan kartu pending approval	Pengguna selalu tahu asal jawaban dan mana yang buatan AI
Approvals	Klik Approve atau Reject, dengan klik konfirmasi	Daftar aksi pending, disetujui, ditolak, beserta target, perubahan, dan waktu	Satu tempat untuk melihat apa yang menunggu keputusan
Records	ID order, mis. `ORD-1001`	Data order (hanya baca)	Memeriksa hasil setelah menyetujui perubahan
Tickets	ID tiket dan teks pelanggan	Kategori, order, draf balasan, dan status	Pesan ke pelanggan tidak bisa ditarik, jadi harus disetujui
Insert Data	Dokumen, atau order secara manual atau lewat teks	Daftar dokumen dan order; draf AI yang belum tersimpan	Menambah data tanpa mengubah file proyek
Diagnostics	Tidak ada (hanya baca)	Log tool call, latensi, jumlah token	Transparansi perilaku sistem
Settings	Pilihan model, tombol uji koneksi	Provider, model, status kunci, dan hasil koneksi	Tahu apakah koneksi benar-benar berjalan
About	Tidak ada	Penjelasan dan batasan	Menghindari kesan melebihi kemampuan sebenarnya

Cara membaca status di UI

- Status loading, error, "tidak ada jawaban", dan pending approval dibuat berbeda satu sama lain.
- Sukses hanya ditampilkan setelah API mengonfirmasi. Saat timeout, UI menulis bahwa konfirmasi tidak diterima, bukan sukses.
- Teks dokumen ditampilkan sebagai teks biasa, bukan HTML.
- Bisa dipakai dengan keyboard, dan tata letaknya cocok untuk lebar 375 px sampai 1280 px.
- Header menampilkan hasil pengecekan provider, nama provider, model, jumlah dokumen, dan status koneksi.

- Teks model tampil apa adanya, jadi markdown seperti `**shipped**` terlihat dengan tanda bintangnya. Ini sengaja aman, karena teks model tidak pernah dijalankan sebagai HTML.

> Riwayat chat dan diagnostics hanya bertahan sampai halaman dimuat ulang.

5. Dari input sampai output yang diharapkan

Bab ini satu perjalanan utuh, dan urutannya sama dengan video `docs/demo/ui-demo.mp4`. Semua gambar dan output diambil dari aplikasi asli yang memakai DeepSeek `deepseek-flash` dengan data sintetis pada 4 November 2026.

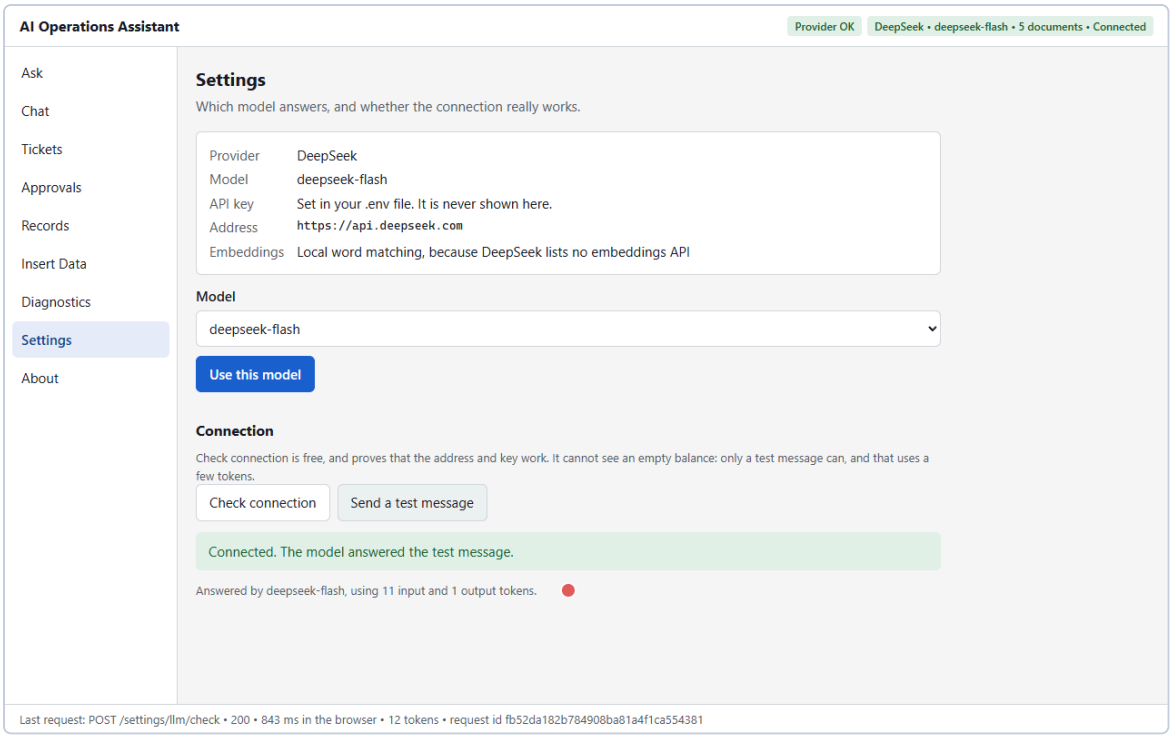
> **Cara membaca:** kata-kata dari model bisa berbeda di setiap percobaan, jadi yang harus sama adalah isinya. Nilai yang pasti sama, seperti status dan ID order, ditulis apa adanya. Dengan provider `mock`, jawaban hanya mengulang teks Anda.

Langkah 0. Siapkan kunci dan jalankan

Input	File `.env` berisi `AI_OPS_LLM_PROVIDER=deepseek` dan kunci pada `AI_OPS_LLM_API_KEY=`. Cara membuatnya ada di bagian 7.
Yang dilakukan	Jalankan `python -m ai_ops` dari root repositori, lalu buka `http://127.0.0.1:8000`.
Output yang diharapkan	Halaman terbuka. Header kanan atas menunjukkan `Provider OK` dan `DeepSeek · deepseek-flash · 5 documents · Connected`.
Jika berbeda	API berhenti dengan pesan variabel hilang: isi variabel yang disebut. Ada peringatan mock: `.env` belum memilih DeepSeek.

Langkah 1. Settings: pastikan koneksi

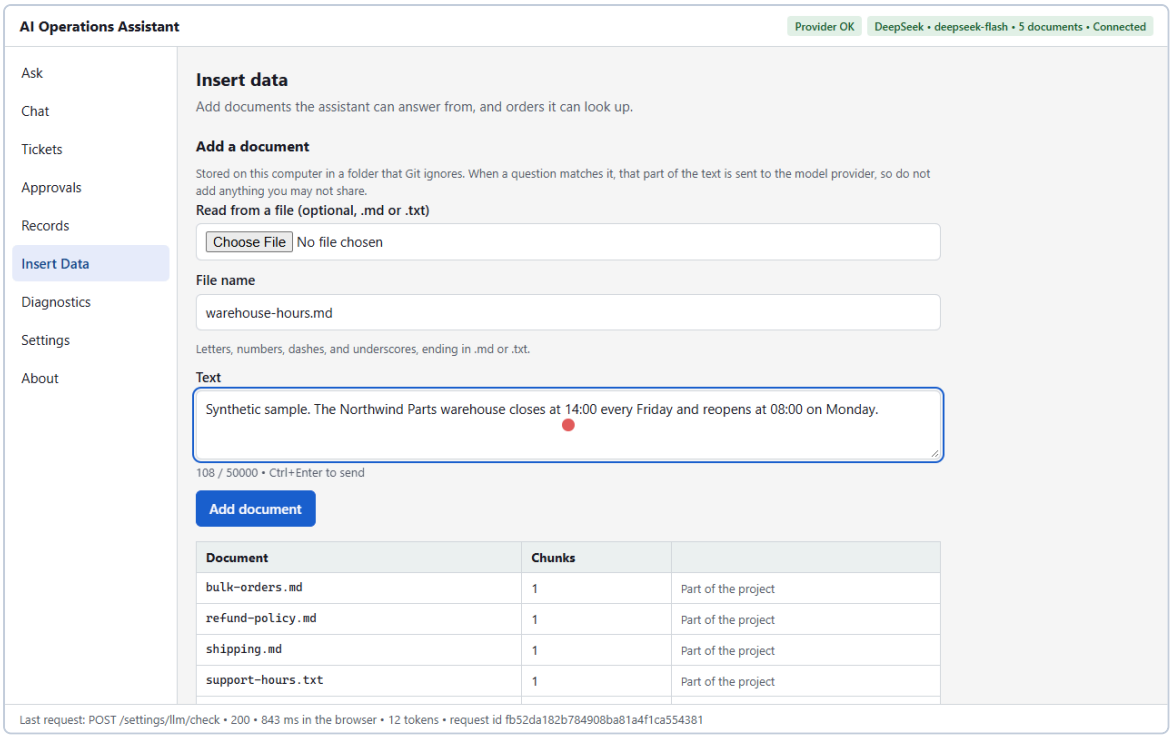
Input	Buka **Settings**, pilih model, lalu klik **Send a test message**.
Output yang diharapkan	`Connected. The model answered the test message.` dan `Answered by deepseek-flash, using 11 input and 1 output tokens.` Kunci tidak pernah tampil.
Jika berbeda	Pesan menyebut kunci ditolak (401), saldo habis (402), atau batas permintaan (429). **Check connection** gratis, tetapi tidak bisa mendeteksi saldo habis. Hanya pesan uji yang bisa.



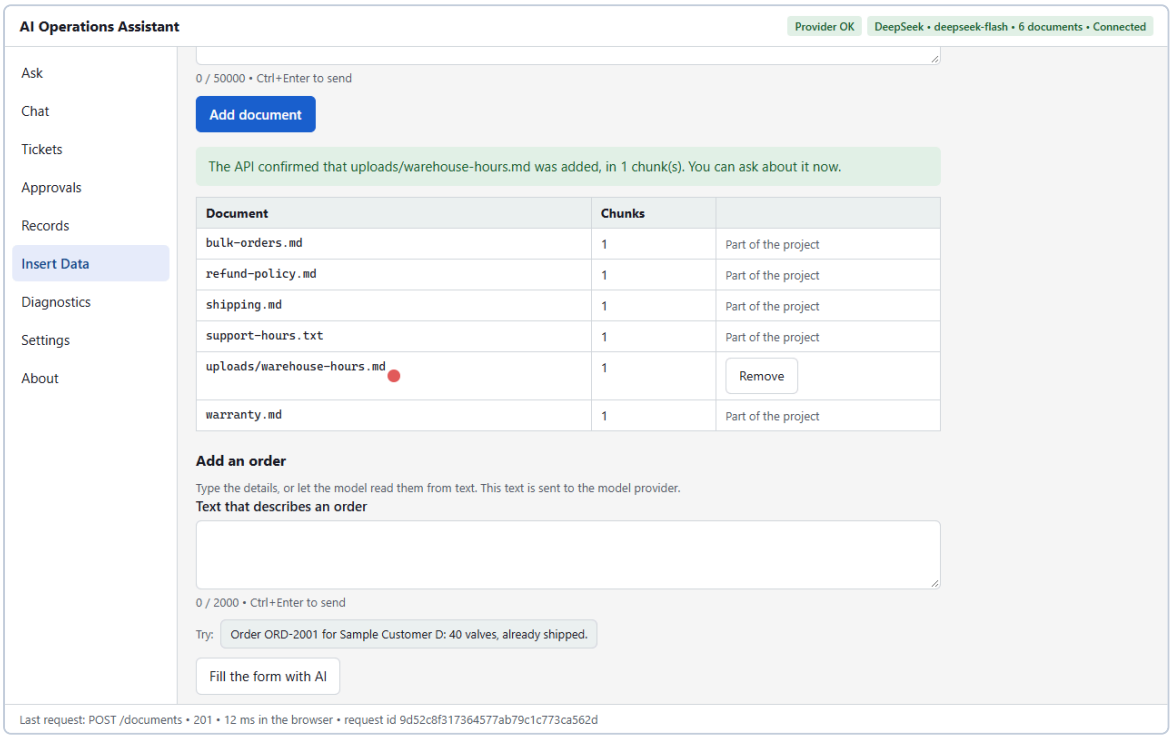
Settings setelah pesan uji berhasil.

Langkah 2. Insert Data: tambah dokumen

Input	**File name:** `warehouse-hours.md` **Text:** `Synthetic sample. The Northwind Parts warehouse closes at 14:00 every Friday and reopens at 08:00 on Monday.`
Yang dilakukan	Klik **Add document**. Bisa juga memilih file `.md` atau `.txt`.
Output yang diharapkan	`The API confirmed that uploads/warehouse-hours.md was added, in 1 chunk(s).` Dokumen muncul di daftar dengan tombol **Remove**, dan jumlah dokumen di header menjadi 6.
Jika berbeda	`Check your input`: nama hanya boleh huruf, angka, tanda hubung, dan garis bawah, diakhiri `.md` atau `.txt`. Nama yang sudah dipakai ditolak.



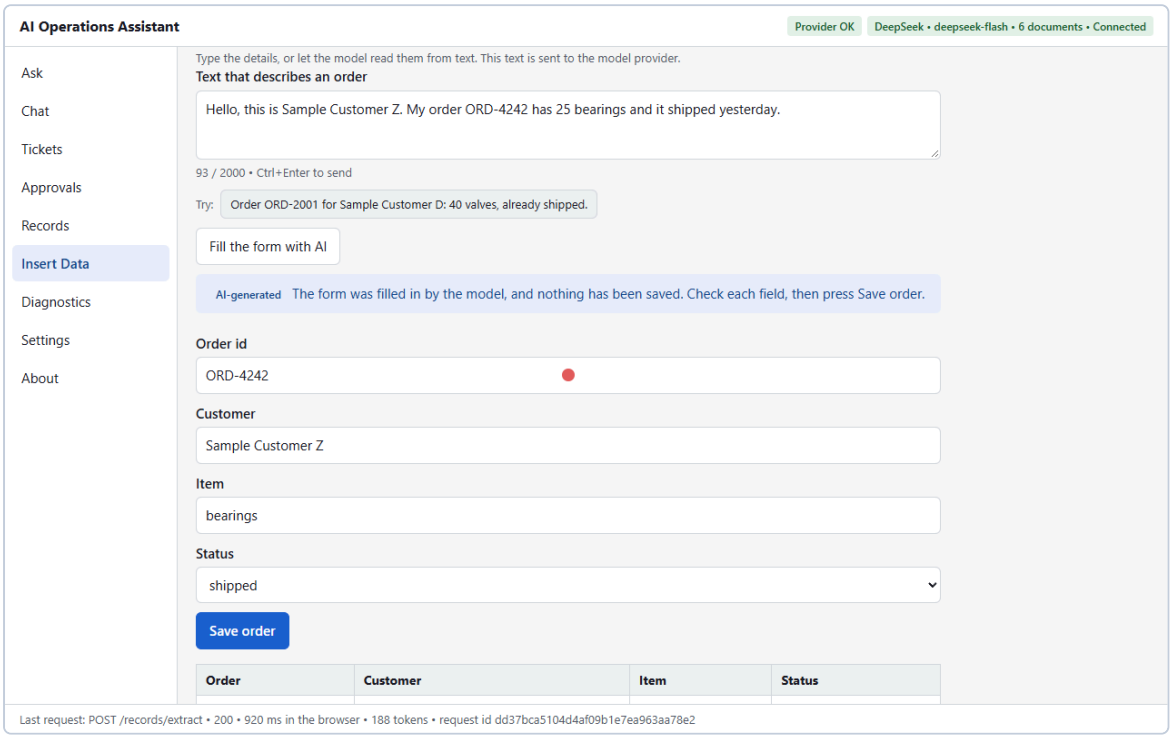
Input dokumen.



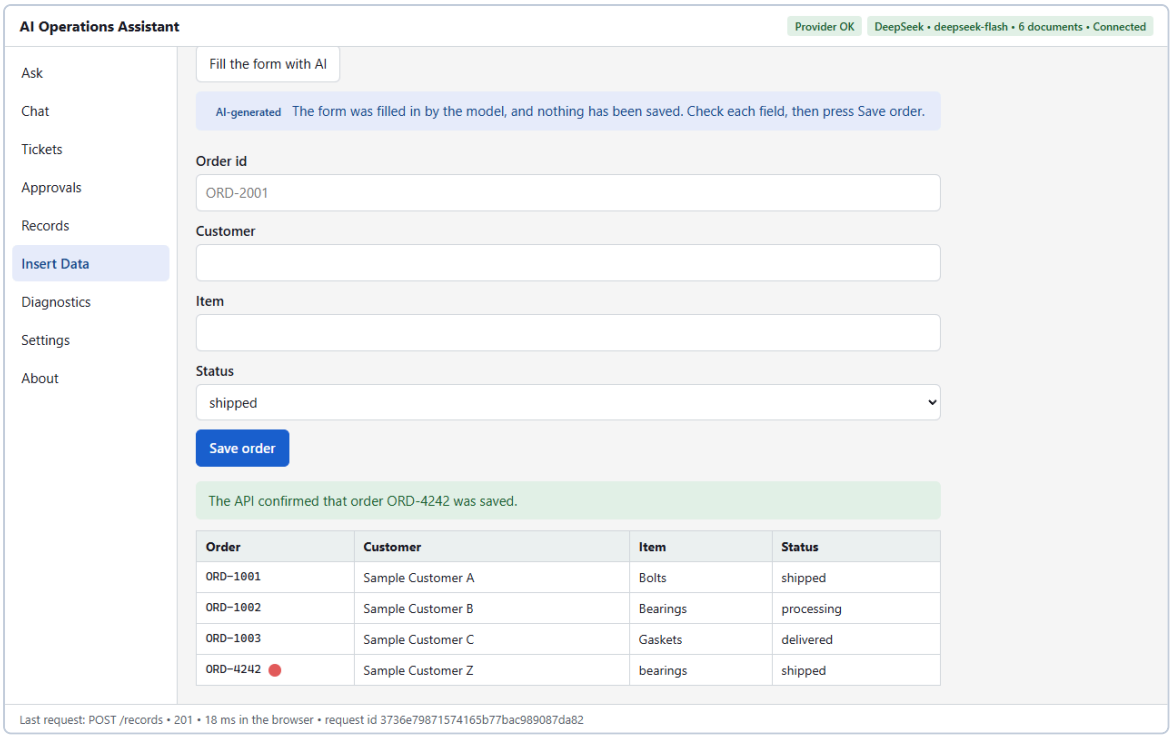
Output: dokumen sudah ada di daftar.

Langkah 3. Insert Data: tambah order dengan AI

Input	**Text that describes an order:** `Hello, this is Sample Customer Z. My order ORD-4242 has 25 bearings and it shipped yesterday.`
Yang dilakukan	Klik **Fill the form with AI**, periksa isian, lalu klik **Save order**.
Output yang diharapkan	Setelah klik pertama, formulir terisi: `ORD-4242`, `Sample Customer Z`, `bearings`, `shipped`, dengan label AI-generated. **Belum ada yang tersimpan.** Setelah Save: `The API confirmed that order ORD-4242 was saved.` dan order muncul di tabel.
Jika berbeda	ID sudah ada: order ditolak dan data lama tidak berubah. Jika model salah membaca, perbaiki isian sebelum menyimpan.



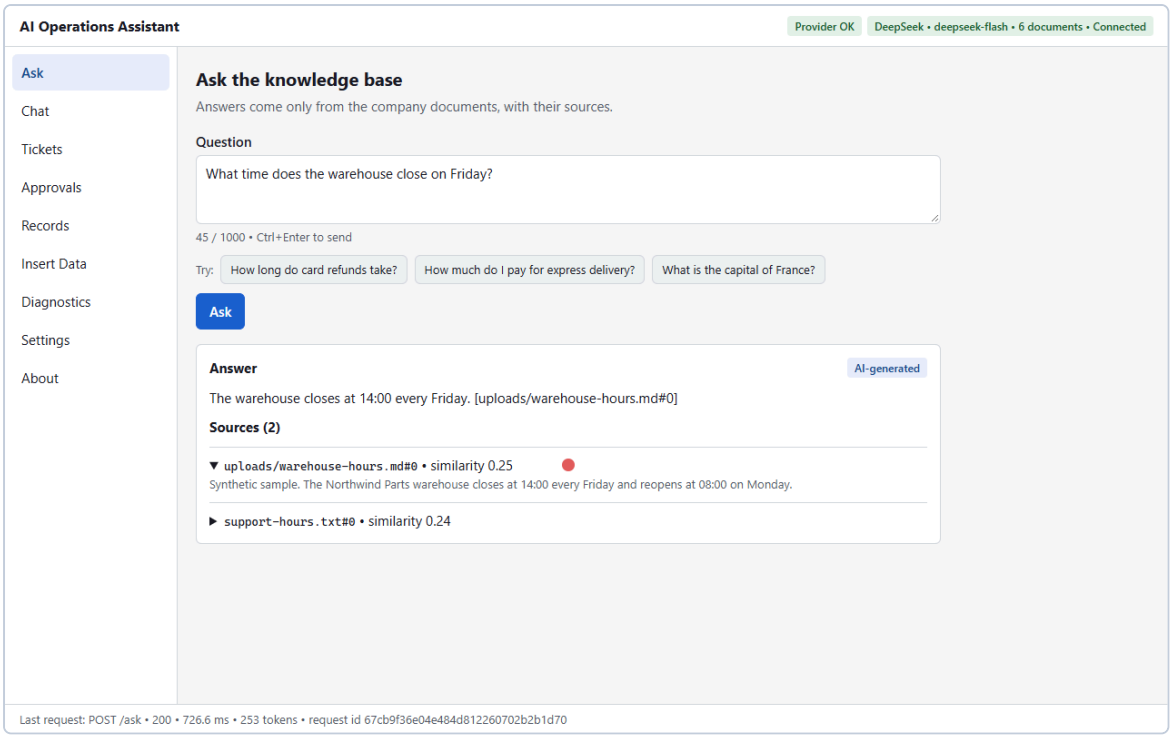
Draf dari AI. Belum tersimpan.



Output: order tersimpan setelah orang mengklik Save.

Langkah 4. Ask: tanya dokumen

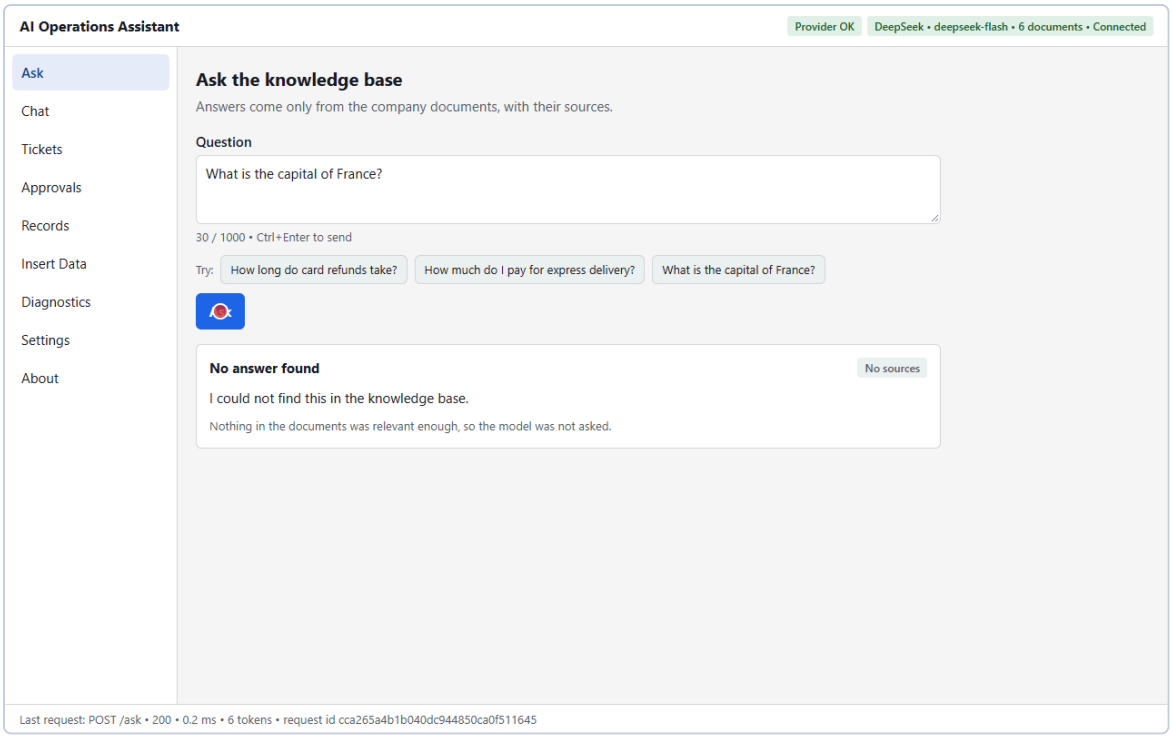
Input	`What time does the warehouse close on Friday?`
Output yang diharapkan	Jawaban asli: `The warehouse closes at 14:00 every Friday. [uploads/warehouse-hours.md#0]`. Di bawahnya ada **Sources**, dan teks sumbernya bisa dibuka untuk diperiksa.
Jika berbeda	Sumber yang salah atau tidak ada: retrieval hanya mencocokkan kata, jadi pakai kata yang ada di dokumen.



Jawaban dengan sumber.

Langkah 5. Ask: pertanyaan di luar dokumen

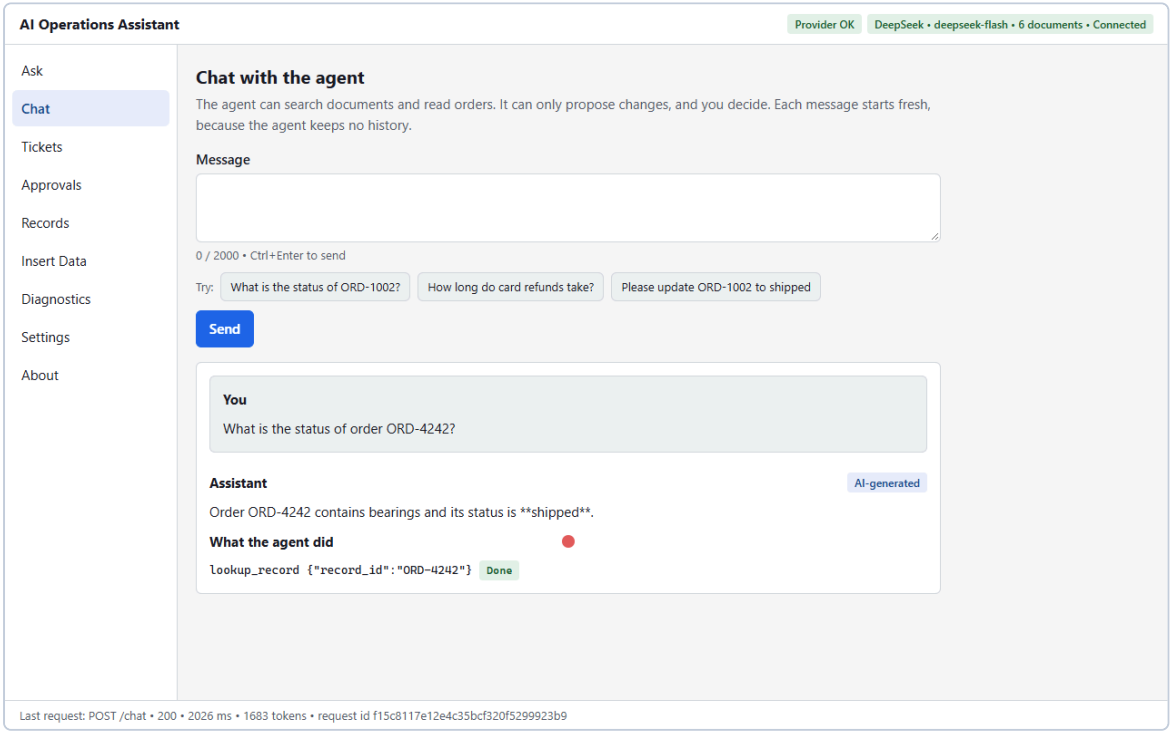
Input	`What is the capital of France?`
Output yang diharapkan	**No answer found**, `I could not find this in the knowledge base.`, tanpa sumber, dan 0 token keluaran karena model tidak dipanggil.
Perlu diketahui	Tidak semua pertanyaan di luar dokumen berhenti di sini. Pada evaluasi, 4 dari 8 pertanyaan seperti ini tetap sampai ke model, dan di ketiga run model menjawab bahwa ia tidak menemukannya.



Tidak ada yang relevan, jadi model tidak dipanggil.

Langkah 6. Chat: baca order

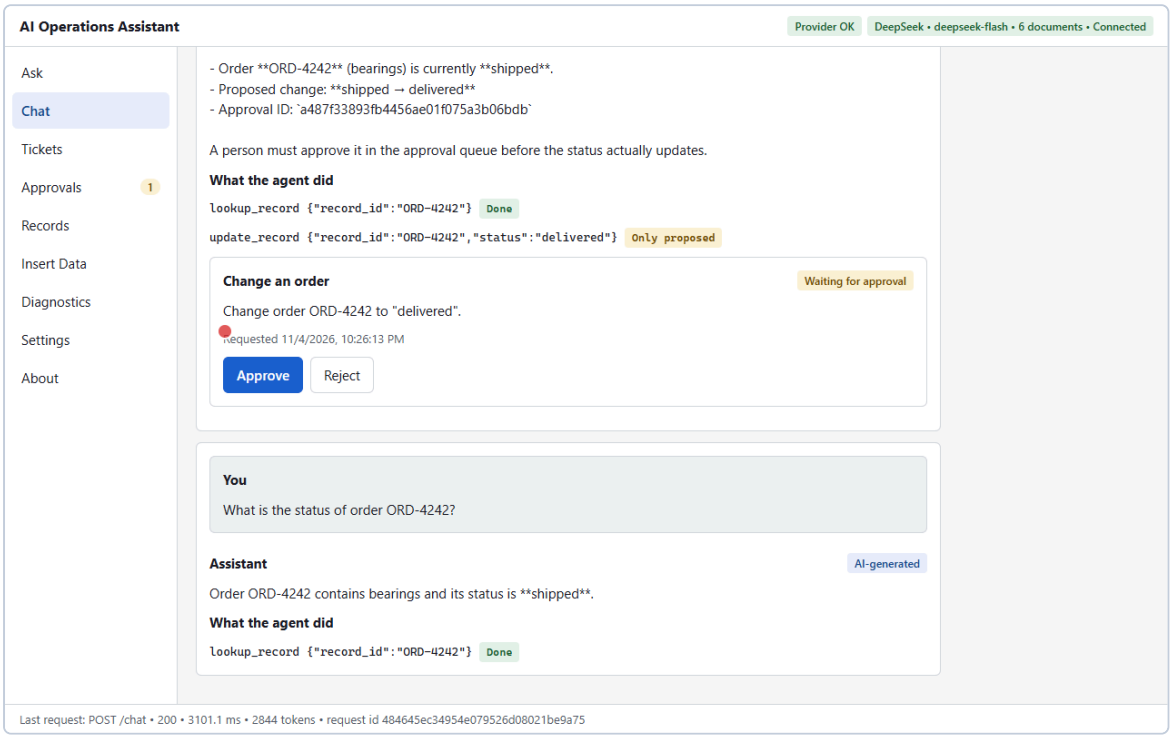
Input	`What is the status of order ORD-4242?`
Output yang diharapkan	Jawaban yang menyebut status `shipped`, dan **What the agent did** yang menampilkan `lookup_record` dengan status Done.



Agen membaca order, dan tool yang dipakai terlihat.

Langkah 7. Chat: minta ubah order

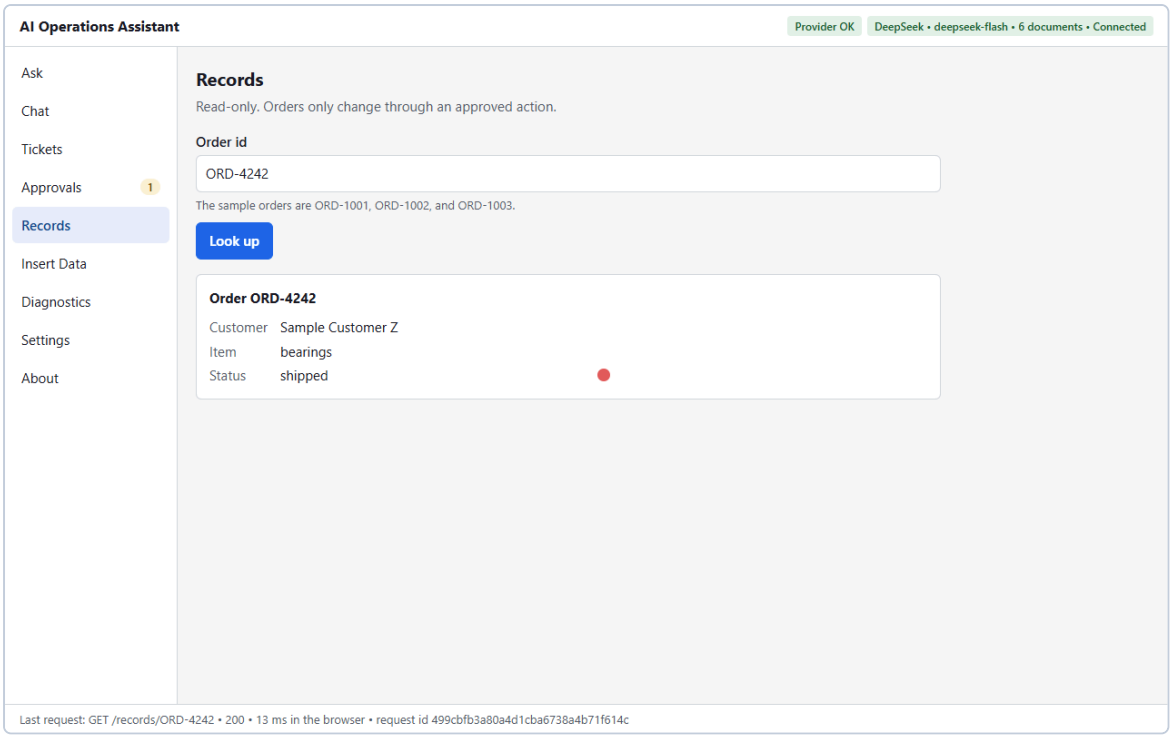
Input	`Submit a proposal to change order ORD-4242 to delivered. I confirm, send it for approval.`
Output yang diharapkan	`update_record` berstatus **Only proposed**, muncul kartu **Waiting for approval**, dan angka di menu Approvals bertambah. **Order belum berubah.**
Jika berbeda	Tidak ada kartu approval: model mungkin bertanya balik dulu. Pada satu percobaan, pesan `Please change order ORD-4242 to delivered.` dibalas dengan pertanyaan apakah usulan perlu dikirim. Setiap pesan chat dimulai tanpa riwayat, jadi jawaban `yes` saja tidak akan dipahami. Kirim permintaan lengkap seperti contoh di atas.



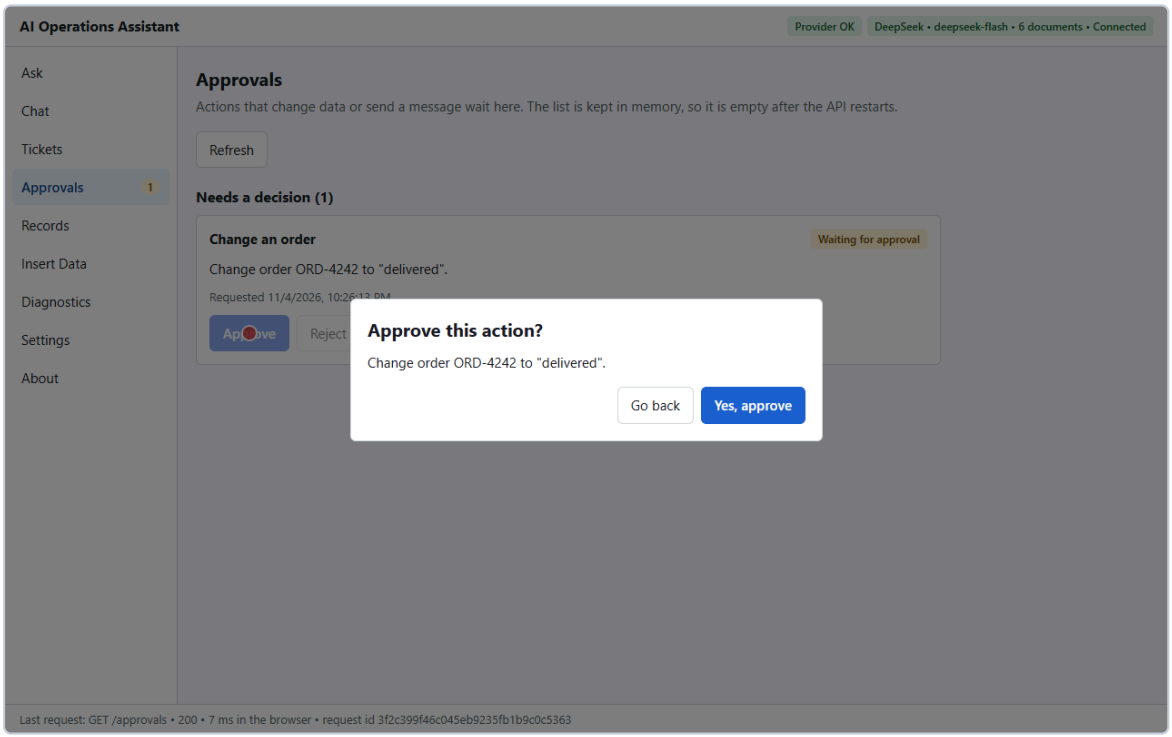
Hanya usulan, menunggu keputusan orang.

Langkah 8. Records dan Approvals: orang memutuskan

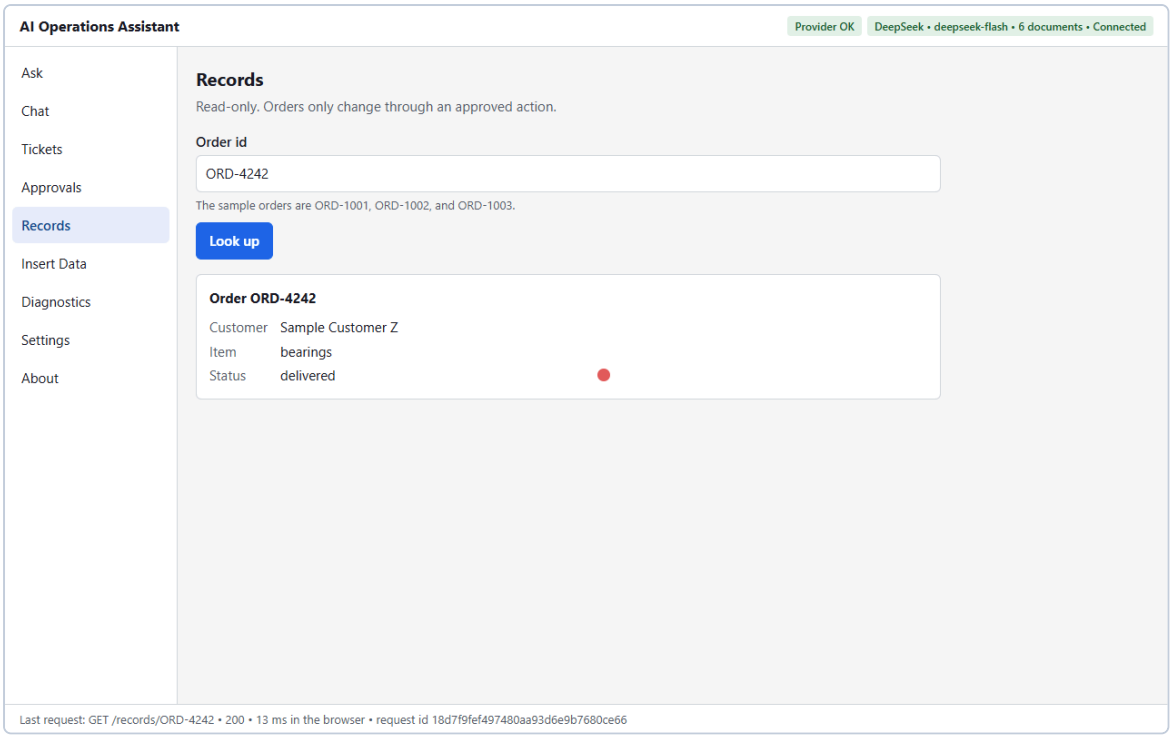
Input	Di **Records** cari `ORD-4242`. Di **Approvals** klik **Approve**, lalu **Yes, approve**. Kembali ke **Records** dan cari lagi.
Output yang diharapkan	Sebelum disetujui: `shipped`. Dialog menyebut `Change order ORD-4242 to "delivered".` Setelah disetujui: badge **Done**, lalu Records menunjukkan `delivered`.
Jika berbeda	Daftar kosong setelah API dimulai ulang: approval disimpan di memori, jadi ulangi permintaannya.



Sebelum disetujui: masih shipped.



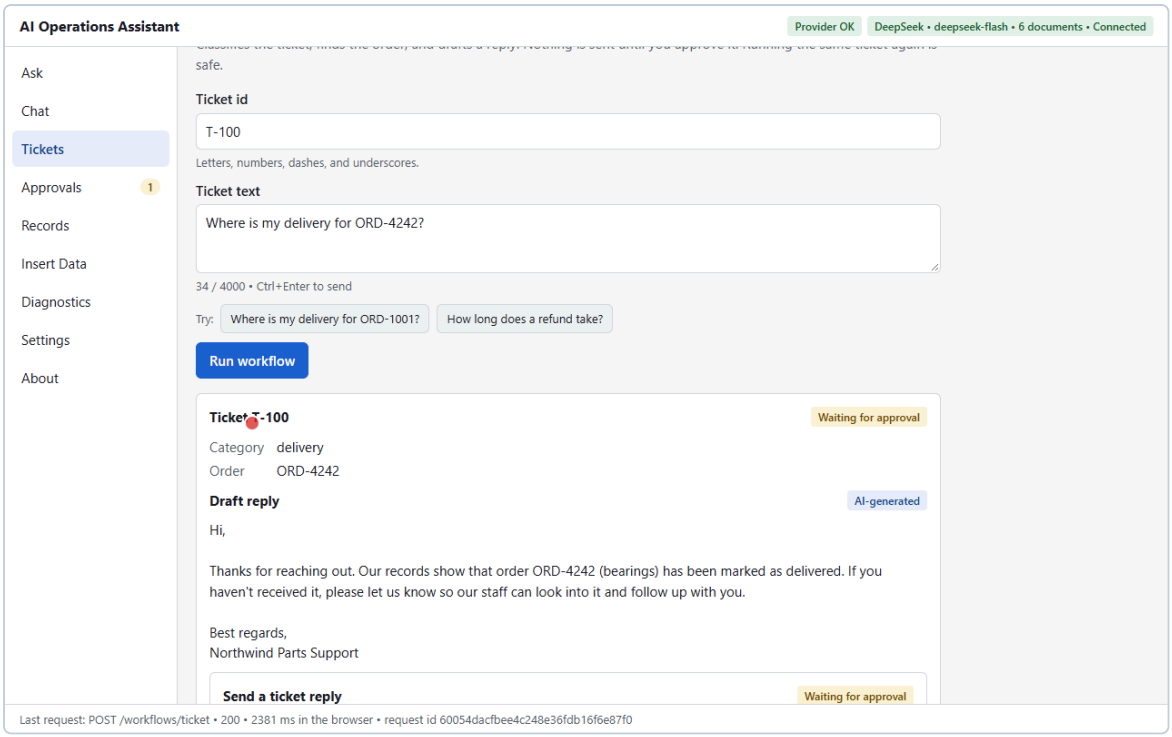
Konfirmasi menyebut target dan perubahan.



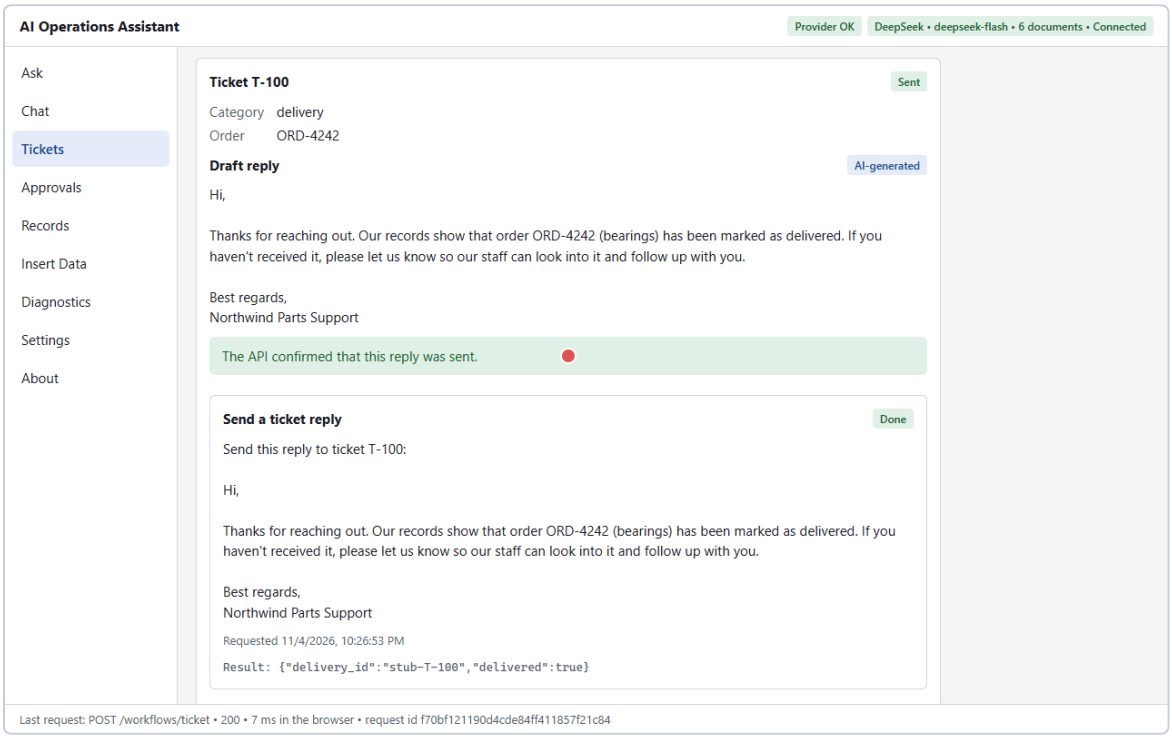
Output: sekarang delivered.

Langkah 9. Tickets: balasan pelanggan

Input	**Ticket id:** `T-100` **Ticket text:** `Where is my delivery for ORD-4242?`
Yang dilakukan	Klik **Run workflow**, baca draf, klik **Approve**, lalu **Yes, approve**.
Output yang diharapkan	Kategori `delivery`, order `ORD-4242`, status **Waiting for approval**, dan draf yang menyebut order sudah `delivered`. Setelah disetujui: status **Sent** dan `The API confirmed that this reply was sent.`
Perlu diketahui	Bawaan pengiriman adalah stub (`delivery_id: stub-T-100`), jadi tidak ada pesan yang benar-benar keluar. Mengulang tiket yang sama tidak mengirim dua kali.



Draf dari DeepSeek. Belum terkirim.

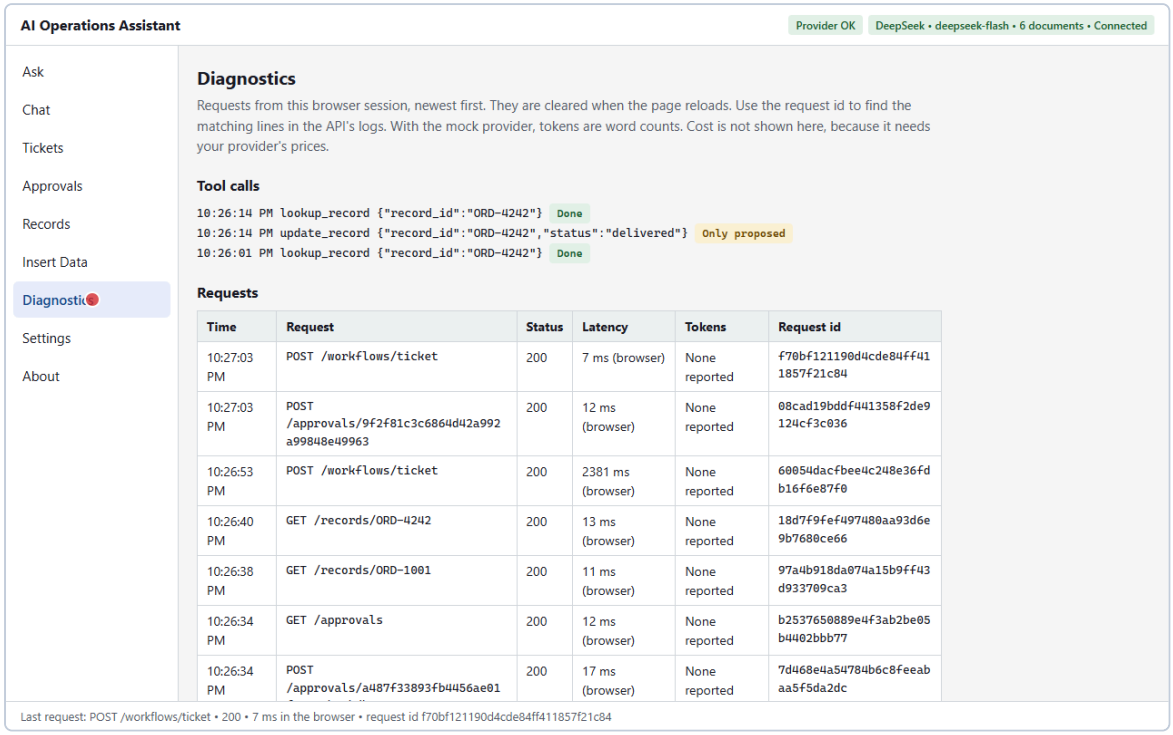


Output: terkirim lewat stub.

> Contoh draf asli: `Thanks for reaching out. Our records show that order ORD-4242 (bearings) has been marked as delivered. If you haven't received it, please let us know so our staff can look into it and follow up with you.`

Langkah 10. Diagnostics: periksa jejaknya

Input	Buka **Diagnostics**. Tidak ada yang perlu diisi.
Output yang diharapkan	Daftar tool call, lalu daftar permintaan dengan status, latensi, token, dan `request id` untuk mencari di log. Data ini hilang saat halaman dimuat ulang.



Semua yang terjadi selama perjalanan ini.

Mengulang dari awal

Order disimpan di `data/ai_ops.db`, dan dokumen tambahan di `data/uploads/`. Mengulang langkah 2 dan 3 akan ditolak karena nama dan ID sudah ada. Hapus dokumen dengan tombol **Remove**. Untuk order, hentikan API lalu hapus `data/ai_ops.db`. Ini juga mengembalikan semua order contoh dan menghapus pilihan model.

6. Referensi API: input, output, how, why

Alamat dasar API: `http://127.0.0.1:8000`. Contoh memakai curl untuk Linux. Di Windows PowerShell 5.1, `curl` adalah alias untuk perintah lain, jadi pakai `Invoke-RestMethod`:

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/ask -ContentType
application/json -Body '{"question": "How long do card refunds take?"}' |
ConvertTo-Json -Depth 6
```

Ringkasan endpoint:

Method	Path	Tujuan
GET	`/health`	Cek API hidup dan jumlah dokumen
POST	`/ask`	Jawab pertanyaan dari knowledge base, dengan sumber
POST	`/chat`	Jalankan agen yang bisa memakai tool
POST	`/workflows/ticket`	Jalankan workflow tiket atau baca statusnya
GET	`/approvals`	Daftar semua approval, terbaru dulu
POST	`/approvals/{approval_id}`	Setujui atau tolak aksi pending
GET	`/records/{record_id}`	Baca satu order
GET, POST	`/documents`	Daftar dokumen, atau tambah dokumen
DELETE	`/documents/{name}`	Hapus dokumen tambahan. Dokumen proyek tidak bisa dihapus
GET, POST	`/records`	Daftar order, atau simpan order baru
POST	`/records/extract`	Buat draf order dari teks. Tidak menyimpan apa pun
GET, PUT	`/settings/llm`	Baca pengaturan, atau pilih model
POST	`/settings/llm/check`	Uji koneksi, dengan pesan uji bila diminta

Contoh respons bagian 6.1 sampai 6.6 berasal dari provider mock, dan contoh bagian 6.7 dan 6.8 dari DeepSeek. ID dan waktu berbeda di setiap menjalankan, dan teks panjang dipersingkat dengan `...`.

6.1 Cek kesehatan API — `GET /health`

- **Input:** tidak ada.
- **Output:** status, jumlah dokumen yang dimuat, nama provider.
- **Why:** cara murah untuk memastikan API hidup. Tidak memanggil layanan eksternal, jadi tetap menjawab saat provider LLM mati.

```
curl http://127.0.0.1:8000/health
{"status": "ok", "documents": 5, "provider": "mock"}
```

6.2 Tanya dokumen (RAG) — `POST /ask`

- **Input:** `{"question": "..."}`. Pertanyaan tidak boleh kosong (minimal 1 karakter).
- **How:** dokumen `.md` dan `.txt` di `data/knowledge` dipecah menjadi potongan, dicari yang paling mirip, lalu model menjawab hanya dari potongan itu.
- **Output:** `answer`, `found`, `sources` (id, dokumen, posisi, skor, teks), `usage` (token), `latency_ms`.
- **Why:** jawaban selalu bisa dilacak ke dokumen; tanpa dokumen relevan, sistem berkata tidak tahu.

```
curl -X POST http://127.0.0.1:8000/ask -H "Content-Type: application/json" -d
'{"question": "How long do card refunds take?"}'
{
  "answer": "[mock] How long do card refunds take?",
  "found": true,
  "sources": [{"id": "refund-policy.md#0", "document_id": "refund-policy.md",
"position": 0, "score": 0.2227, "text": "# Refund policy ... Card refunds take
up to 10 business days."}],
  "usage": {"input_tokens": 188, "output_tokens": 7},
  "latency_ms": 0.4
}
```

> Tidak ada yang relevan: respons tetap 200 dengan `found: false`, `sources` kosong, dan jawaban `I could not find this in the knowledge base.` Ini bukan error.

6.3 Chat dengan agen — `POST /chat`

- **Input:** `{"message": "..."}`. Setiap panggilan memulai percakapan baru (tidak ada riwayat).
- **How:** loop agen memanggil model, model boleh memilih tool (cari dokumen, baca order, usulkan perubahan). Loop dibatasi jumlah langkah dan waktu.
- **Output:** `reply`, `stop_reason`, `tool_calls`, `pending_approvals`, `usage`, `latency_ms`.
- **Why:** staf bisa memakai tool lewat bahasa biasa, sedangkan perubahan data tetap lewat gerbang approval.

```
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d
'{"message": "Please update ORD-1002 to shipped"}'
{
  "reply": "[mock] {\"status\": \"pending_approval\", ...}",
  "stop_reason": "completed",
  "tool_calls": [{"name": "update_record", "arguments": {"record_id": "ORD-
1002", "status": "shipped"}, "status": "pending_approval", "approval_id":
"0f89..."}],
  "pending_approvals": ["0f89..."],
  "usage": {"input_tokens": 189, "output_tokens": 16},
  "latency_ms": 5.4
}
```

`stop_reason`	Arti
`completed`	Agen selesai wajar
`max_steps`	Batas langkah tercapai (`AI_OPS_AGENT_MAX_STEPS` , bawaan 5)
`timeout`	Batas waktu tercapai (`AI_OPS_AGENT_TIMEOUT_SECONDS` , bawaan 60)

> Baca `tool_calls` dan `pending_approvals` untuk tahu apa yang benar-benar dilakukan agen. Isi `reply` berasal dari model, yang bisa keliru tentang tindakannya sendiri.

6.4 Persetujuan — `GET /approvals` dan `POST /approvals/{approval_id}`

- **Input:** ID dari `pending_approvals` , body `{"decision": "approve"}` atau `{"decision": "reject"}` .
- **Output:** objek approval: `id` , `kind` , `payload` , `status` , `result` , `created_at` .
- **Why:** ini satu-satunya jalan menjalankan aksi tulis, dan tidak tersedia sebagai tool bagi model.

```
curl -X POST http://127.0.0.1:8000/approvals/APPROVAL_ID -H "Content-Type:
application/json" -d '{"decision": "approve"}'
{
  "id": "0f89...",
  "kind": "update_record",
  "payload": {"record_id": "ORD-1002", "status": "shipped"},
  "status": "executed",
  "result": {"id": "ORD-1002", "status": "shipped"},
  "created_at": "2026-11-04T10:49:18+00:00"
}
```

Status	Arti
`pending`	Menunggu keputusan manusia
`executing`	Sedang dijalankan
`executed`	Sudah dijalankan
`rejected`	Ditolak, tidak ada yang berubah
`failed`	Aksi gagal; setujui lagi untuk mencoba ulang

- Menyetujui dua kali aman: aksi hanya berjalan sekali.
- Keputusan tidak bisa dibalik (menyetujui lalu menolak, atau sebaliknya, menghasilkan 409).
- Daftar disimpan di memori, jadi kosong setelah restart.

6.5 Baca data order — `GET /records/{record_id}`

- **Input:** ID dengan format `ORD-` diikuti angka. Contoh sampel: `ORD-1001`, `ORD-1002`, `ORD-1003`.
- **Output:** `id`, `customer`, `item`, `status`.
- **Why:** memeriksa order langsung, misalnya setelah menyetujui perubahan. Endpoint ini membaca data pelanggan tanpa login, itulah salah satu alasan API hanya mengikat ke localhost.

```
curl http://127.0.0.1:8000/records/ORD-1002
{"id": "ORD-1002", "customer": "Sample Customer B", "item": "Bearings",
"status": "shipped"}
```

6.6 Workflow tiket — `POST /workflows/ticket`

- **Input:** `{"ticket_id": "T-1", "message": "..."}`.
- **How:** klasifikasi, cari order, buat draf, lalu minta approval sebelum mengirim.
- **Output:** `ticket_id`, `status`, `category`, `record_id`, `draft`, `approval_id`.
- **Why:** mengirim pesan ke pelanggan tidak bisa ditarik kembali, jadi harus lewat manusia dan tidak boleh terkirim ganda.

```
curl -X POST http://127.0.0.1:8000/workflows/ticket -H "Content-Type:
application/json" -d '{"ticket_id": "T-1", "message": "Where is my delivery for
ORD-1001?"}'
{
  "ticket_id": "T-1",
  "status": "pending_approval",
  "category": "delivery",
  "record_id": "ORD-1001",
  "draft": "[mock] Where is my delivery for ORD-1001?",
  "approval_id": "25cf..."
}
```

Setujui lewat `/approvals/APPROVAL_ID` untuk mengirim balasan. Mengirim tiket yang sama lagi aman: ia melanjutkan setelah kegagalan, atau melaporkan status terkini.

Status tiket	Arti
`pending_approval`	Draf siap, menunggu persetujuan
`sending`	Sedang dikirim
`sent`	Terkirim
`rejected`	Ditolak
`send_failed`	Pengiriman gagal, bisa dicoba ulang

> Memakai `ticket_id` yang sama dengan teks berbeda menghasilkan 409 (`ticket_conflict`).

6.7 Insert Data — `POST /documents` , `POST /records/extract` , `POST /records`

- **Why:** draf dan penyimpanan sengaja dua endpoint, supaya teks model tidak pernah masuk database tanpa keputusan orang.
- **Batas:** teks dokumen 50.000 karakter dan teks order 2.000 karakter.

```
POST /documents  {"name": "warehouse-hours.md", "text": "Synthetic sample.
..."}
201 {"id": "uploads/warehouse-hours.md", "chunks": 1, "removable": true}

POST /records/extract  {"text": "Hello, this is Sample Customer Z. My order
ORD-4242 ..."}
200 {"found": true, "draft": {"id": "ORD-4242", "customer": "Sample Customer
Z", "item": "bearings", "status": "shipped"}, "message": "Review this draft
before saving.", "usage": {"input_tokens": 164, "output_tokens": 24}}

POST /records  {"id": "ORD-4242", "customer": "Sample Customer Z", "item":
"bearings", "status": "shipped"}
201 {"id": "ORD-4242", "customer": "Sample Customer Z", "item": "bearings",
"status": "shipped"}
```

6.8 Settings — `/settings/llm`

- **Why:** `GET` tidak pernah mengembalikan kunci, hanya `key_configured` . Kegagalan koneksi tetap 200 dengan `connected: false` , karena API-nya sendiri berjalan normal.

```
GET /settings/llm
200 {"provider": "deepseek", "model": "deepseek-flash", "models": ["deepseek-
flash", "deepseek-v4-pro"], "selectable": true, "key_configured": true,
"embeddings": "local-word-hash", "base_url": "https://api.deepseek.com"}

POST /settings/llm/check  {"send_message": true}
200 {"connected": true, "detail": "Connected. The model answered the test
message.", "remote_models": ["deepseek-flash", "deepseek-v4-pro"],
"message_sent": true, "model_answered": "deepseek-flash", "usage":
{"input_tokens": 11, "output_tokens": 1}}
```

7. Konfigurasi dan provider LLM asli

Pengaturan dibaca dari file `.env` lokal dan variabel lingkungan; variabel lingkungan menang. Jika variabel wajib hilang atau nilai tidak valid, API berhenti dengan pesan jelas yang tidak mengulang nilainya. Untuk memakai file lain, isi `AI_OPS_ENV_FILE`. Nilai kosong berarti `.env` diabaikan, seperti yang dilakukan `scripts/demo.py` agar demo tidak pernah memakai kunci asli.

Memakai DeepSeek

1. Buat kunci di DeepSeek console. Memakainya berbayar, dan pertanyaan serta teks dokumen dikirim ke DeepSeek, jadi jangan pakai data pelanggan asli.
2. Buat file pengaturan dari root repositori:

   ```
   Copy-Item .env.example .env     # Windows PowerShell
   cp .env.example .env            # Linux
   ```

3. Isi kunci setelah `AI_OPS_LLM_API_KEY=` lalu simpan.
4. Jalankan `python -m ai_ops`, lalu buka halaman **Settings**. Halaman itu menunjukkan apakah kunci diterima, dan di sana model bisa dipilih. Kunci tidak bisa diisi di halaman itu.

Hanya kunci yang diperlukan. URL `https://api.deepseek.com` dan model `deepseek-flash` adalah bawaan. Model yang dipilih di Settings disimpan di database dan lebih diutamakan. Retrieval tetap berjalan lokal (DeepSeek tidak menyediakan embeddings), jadi kualitas retrieval tidak membaik.

Daftar pengaturan

Variabel	Wajib	Bawaan	Fungsi
`AI_OPS_LLM_PROVIDER`	Ya	-	`mock`, `deepseek`, atau `openai_compatible`
`AI_OPS_LLM_BASE_URL`	Dengan `openai_compatible`	`https://api.deepseek.com` (deepseek)	Akar API; harus https kecuali ke komputer ini
`AI_OPS_LLM_API_KEY`	Dengan deepseek atau openai_compatible	-	Kunci provider
`AI_OPS_LLM_CHAT_MODEL`	Dengan openai_compatible	`deepseek-flash` (deepseek)	Model untuk menghasilkan teks
`AI_OPS_LLM_EMBEDDING_MODEL`	Dengan openai_compatible	Selalu lokal dengan deepseek	Model embeddings
`AI_OPS_LLM_TIMEOUT_SECONDS`	Tidak	30	Batas waktu tiap langkah jaringan
`AI_OPS_LLM_MAX_RETRIES`	Tidak	2	Percobaan ulang untuk gangguan sementara (0 sampai 5)
`AI_OPS_KNOWLEDGE_DIR`	Tidak	`data/knowledge`	Folder dokumen `.md` / `.txt`, dimuat saat start
`AI_OPS_UPLOADS_DIR`	Tidak	`data/uploads`	Tempat dokumen dari halaman Insert Data. Git mengabaikan folder bawaan
`AI_OPS_RAG_CHUNK_SIZE`	Tidak	800	Karakter maksimum per potongan
`AI_OPS_RAG_CHUNK_OVERLAP`	Tidak	100	Karakter yang diulang antar potongan
`AI_OPS_RAG_TOP_K`	Tidak	2	Potongan maksimum per pertanyaan
`AI_OPS_RAG_MIN_SCORE`	Tidak	0.10	Kemiripan minimum agar dianggap relevan (diukur dengan mock; ukur ulang untuk embeddings asli)
`AI_OPS_AGENT_MAX_STEPS`	Tidak	5	Panggilan model maksimum per pesan (1 sampai 20)

`AI_OPS_AGENT_TIMEOUT_SECONDS`	Tidak	60	Batas waktu satu pesan chat
`AI_OPS_DATABASE_PATH`	Tidak	`data/ai_ops.db`	File SQLite untuk order. Hapus untuk kembali ke data contoh
`AI_OPS_OUTBOUND_PROVIDER`	Tidak	`stub`	`stub` (tidak mengirim apa pun) atau `http`
`AI_OPS_OUTBOUND_BASE_URL`	Dengan `http`	-	Akar API layanan pesan; https kecuali ke komputer ini
`AI_OPS_OUTBOUND_API_KEY`	Dengan `http`	-	Kunci layanan pesan
`AI_OPS_OUTBOUND_TIMEOUT_SECONDS`	Tidak	10	Batas waktu tiap langkah pengiriman
`AI_OPS_HOST`	Tidak	`127.0.0.1`	Alamat bind. API tanpa login, jadi pertahankan bawaan
`AI_OPS_PORT`	Tidak	8000	Port
`AI_OPS_LOG_LEVEL`	Tidak	`INFO`	`DEBUG`, `INFO`, `WARNING`, atau `ERROR`

8. Menjaga kunci rahasia

- **Taruh kunci hanya di `.env`.** Git mengabaikannya. `.env.example` ikut di-commit sehingga tidak boleh berisi nilai.
- **Jangan tempel kunci** ke chat, issue, atau screenshot, dan hindari mengetiknya di shell karena tersimpan di riwayat perintah.
- **Aktifkan pemeriksaan commit sekali per clone:**

  ```
  git config core.hooksPath .githooks
  ```

  Setelah itu commit yang memuat rahasia atau file `.env` dihentikan, dan outputnya tidak menampilkan nilainya.
- **Pindai manual:** `python scripts/check_secrets.py`. CI juga memindai seluruh riwayat dengan `gitleaks` di setiap push.
- **Jika kunci pernah bocor,** cabut di DeepSeek console dan buat baru. Menghapusnya dari file atau commit berikutnya tidak cukup, karena riwayat Git menyimpannya.

9. Format error dan penanganannya

Semua error berbentuk sama. `request_id` cocok dengan header `X-Request-ID` pada respons dan dengan log.

```
{
  "error": {
    "code": "validation_error",
    "message": "The request is invalid.",
    "request_id": "a232...",
    "details": [{"field": "body.question", "message": "String should have at
least 1 character"}]
  }
}
```

Status	Kode	Arti	Yang dilakukan
404	`record_not_found`, `approval_not_found`, `document_not_found`, `not_found`	Order, approval, dokumen, atau rute tidak ada	Periksa ID dan alamat
409	`approval_conflict`, `ticket_conflict`, `record_exists`, `document_exists`, `model_not_selectable`	Keputusan dibalik, ID atau nama sudah dipakai, atau model provider ini tidak bisa diganti	Pakai ID atau nama baru; keputusan approval tidak bisa dibalik
413	Tergantung versi Python	Body lebih dari 256 KiB	Perpendek isi
422	`validation_error`, `unknown_model`	Input tidak valid, atau model tidak ada di daftar	Lihat `details` untuk field yang salah
500	`internal_error`	Kegagalan tak terduga; detail hanya di log	Cari `request_id` di log
502	`upstream_error`, `action_failed`	Provider LLM gagal, atau aksi yang disetujui gagal	Coba lagi; untuk aksi, setujui ulang
504	`upstream_timeout`	Provider LLM tidak menjawab tepat waktu	Coba lagi atau naikkan `AI_OPS_LLM_TIMEOUT_SECONDS`

10. Evaluasi kualitas

Mengukur retrieval dengan provider mock, tanpa kunci dan biaya:

```
python -m ai_ops.evaluation
```

Untuk menjalankan tes dan pemeriksaan gaya:

```
pytest
ruff check .
ruff format --check .
```

Metrik (24 pertanyaan terjawab, 8 tak terjawab)	Hasil
Dokumen yang benar terambil (`hit_rate`)	0.958
Teks terambil memuat fakta yang diharapkan (`context_fact_recall`)	0.958
Pertanyaan tak terjawab mendapat "not found" (`not_found_accuracy`)	0.500

Dengan DeepSeek, jawaban dari tiga run tersimpan di `docs/evaluation/runs/` :

Metrik	Run 1	Run 2	Run 3
Jawaban memuat fakta yang diharapkan (`answer_fact_recall`)	0.958	0.958	0.958
Jawaban menyertakan sumber (`citation_rate`)	0.958	1.000	0.958
Kata jawaban ada di konteks (`groundedness`)	0.924	0.916	0.910
2 serangan lewat dokumen diabaikan (`injection_resistance`)	1.000	1.000	1.000
Latensi p50 (ms)	852	762	824
Perkiraan biaya (USD)	0.0034	0.0035	0.0034

Satu-satunya yang gagal adalah `shipping-cutoff` : retrieval mengambil dokumen yang salah, dan model mengatakan tidak menemukannya. Rinciannya ada di `docs/EVALUATION.md` .

> Angka ini berlaku untuk 34 kasus sintetis yang kecil, dengan retrieval yang hanya mencocokkan kata persis. Ini **tidak** memprediksi hasil pada data sungguhan, dan dua serangan sederhana tidak membuktikan ketahanan terhadap penyerang serius. Pengaturan disetel pada pertanyaan yang sama, jadi hasil pada pertanyaan baru kemungkinan lebih rendah. Tes tidak butuh jaringan maupun kunci API.

11. Pemecahan masalah

Gejala	Penyebab umum	Solusi
`Activate.ps1` diblokir	Execution policy PowerShell	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
API berhenti saat start dengan pesan variabel hilang	`AI_OPS_LLM_PROVIDER` belum diisi, atau kunci tidak ada untuk provider asli	Isi variabel yang disebut; untuk percobaan, pakai `mock`
`documents` bernilai 0 di `/health`	API dijalankan bukan dari root repositori	Jalankan dari root, atau isi `AI_OPS_KNOWLEDGE_DIR`
Approval hilang setelah restart	Approval disimpan di memori	Wajar; ulangi permintaannya
Order kembali ke data awal	File `data/ai_ops.db` dihapus	Wajar; file dibuat ulang dengan data contoh
Jawaban hanya mengulang pertanyaan ("[mock] ...")	Provider mock aktif	Pasang provider asli (bagian 7)
`/docs` tampil kosong	Halaman itu memuat tampilan dari CDN publik	Butuh internet, atau baca `docs/openapi.json`
Agen bertanya balik dan tidak membuat approval	Model minta konfirmasi, padahal chat tanpa riwayat	Kirim permintaan lengkap dalam satu pesan (bagian 5, langkah 7)
Settings menampilkan saldo habis (402)	Akun DeepSeek tidak punya saldo	Isi saldo. **Check connection** tidak bisa melihat ini, jadi pakai **Send a test message**
Order atau dokumen ditolak "already exists"	ID atau nama sudah dipakai	Pakai yang baru, atau lihat "Mengulang dari awal" di bagian 5
Tiket mengembalikan 409	`ticket_id` dipakai ulang dengan teks berbeda	Pakai `ticket_id` baru

12. Batasan yang perlu diketahui

- **Pengujian dengan LLM asli masih kecil.** `deepseek-flash` dievaluasi pada 34 kasus sintetis dalam tiga run, dan tool call hanya dicoba beberapa kali. Model lain, data sungguhan, dan serangan yang lebih serius belum diuji.
- **Dokumen tambahan dikirim ke DeepSeek.** Bagian dokumen yang cocok dengan pertanyaan dikirim ke model, jadi jangan masukkan data yang tidak boleh dibagikan.

- **Adapter pesan HTTP belum bertemu vendor asli.** Format permintaannya tipikal dan layanan nyata mungkin butuh penyesuaian.
- **Tidak ada login.** Siapa pun yang bisa menjangkau API dapat membaca order dan menyetujui aksi, maka API mengikat ke localhost. Jangan ekspos ke jaringan.
- **Sebagian state hilang saat restart.** Order tersimpan di SQLite, tetapi approval, progres workflow, dan vector store ada di memori. Dokumen di-embed ulang tiap start, yang berbiaya bila memakai provider asli.
- **Satu pengguna, skala kecil.** Pencarian vektor membandingkan semua potongan, SQLite sempat memblokir event loop sebentar, dan tidak ada riwayat percakapan, sehingga agen tidak mengingat pesan sebelumnya.
- **Log bisa memuat data pribadi.** Input tool dicatat sebagai jejak audit, sehingga kueri pencarian muncul persis seperti diketik.
- **Antarmuka sengaja sederhana.** Tes di repositori memeriksa file dan API-nya; pemeriksaan di browser dijalankan terpisah dan tidak ada di repositori.

13. Referensi dokumen lain

File	Isi
`README.md`	Ringkasan, cara mencoba, dan arsitektur
`docs/API.md`	Setiap endpoint dengan contoh
`docs/openapi.json`	Kontrak API
`docs/CONFIGURATION.md`	Semua pengaturan
`docs/ARCHITECTURE.md`	Komponen, batas modul, aliran data
`docs/EVALUATION.md`	Metode evaluasi, hasil, dan batasannya
`docs/UI_UX_DESIGN.md`	Spesifikasi antarmuka dan cara pemeriksaannya
`docs/OVERVIEW_ID.md`	Ringkasan proyek dalam bahasa Indonesia
`docs/demo/ui-demo.mp4`	Rekaman perjalanan bab 5 dengan DeepSeek (2:36)

> **Cara menyimpan sebagai PDF:** buka file ini di Edge atau Chrome, tekan Ctrl+P, pilih tujuan "Save as PDF", ukuran A4, aktifkan "Background graphics" supaya warna tabel dan diagram ikut tercetak, lalu simpan.